Service-Oriented Architecture (SOA)

Service-Oriented Architecture: According to the World Wide Web Consortium (W3C), SOA is “a methodology for achieving application interoperability and reuse of IT assets that feature a strong architectural focus, an ideal level of abstraction, a deployment infrastructure, and a reusable library of services.”

W3C describes SOA implementations as logical abstracts of the services component infrastructure, which views services as formally defined in terms of messages exchanged between providers and requesters.

Physically, services are characterized as message-based exchanges that are network-oriented, granular (a small number of operations and large messages), platform-neutral, and described by processable metadata.

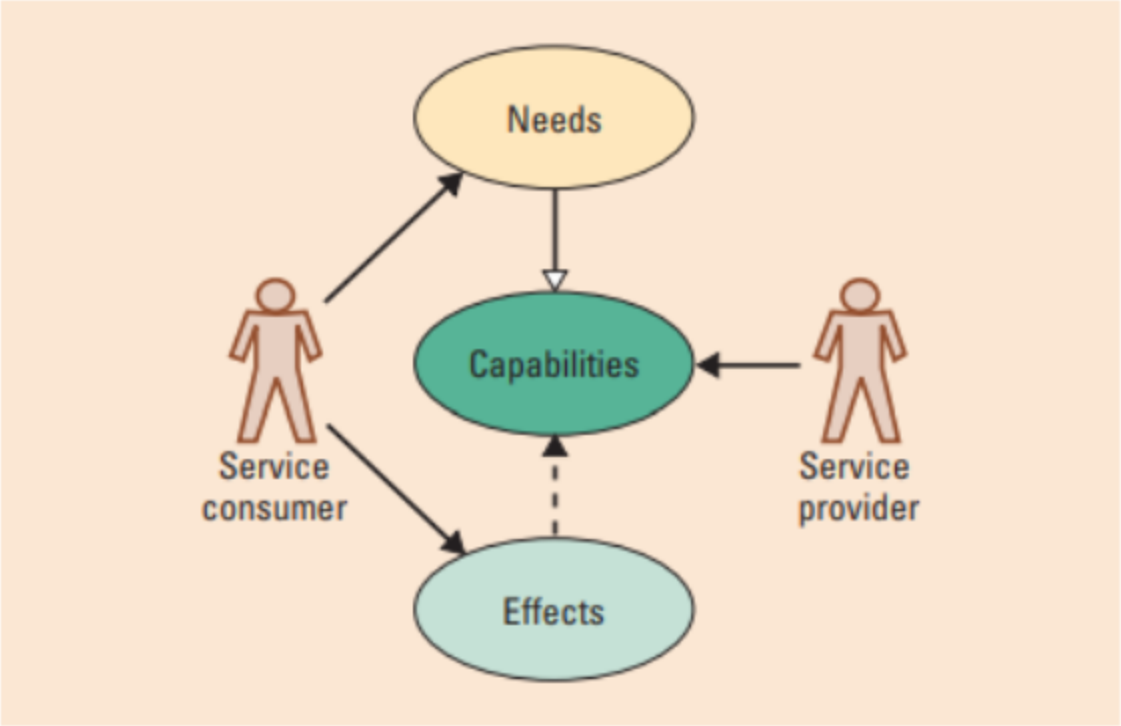

Figure 1. A Unified Modeling Language use case diagram of a SOA requester–provider interaction. The requester gains effects from the service provider on the basis of matched capabilities.

-Visibility refers to the provider’s and requester’s ability to interact and accomplish the requested service capability. This dimension introduces opportunities for matching the requester’s needs to the provider’s capabilities and vice versa (see Figure 1). We can decompose visibility according to related attributes or functional requirements,
such as awareness, willingness, reachability, and self-serviceability

-Interaction—when the requester uses one of the provider's capabilities—is mediated through message exchange and produces an effect (or set of effects). An effect can take the form of information resulting from using a capability, a state change in entities (defined or undefined), or a combination of both. Here, we define a service as a mechanism that lets a requester (consumer) use predefined capabilities.

A service interface specifies how to access a service's capabilities. For a service to achieve visibility, the service interface of its information and behavior models must be exposed to requesters.

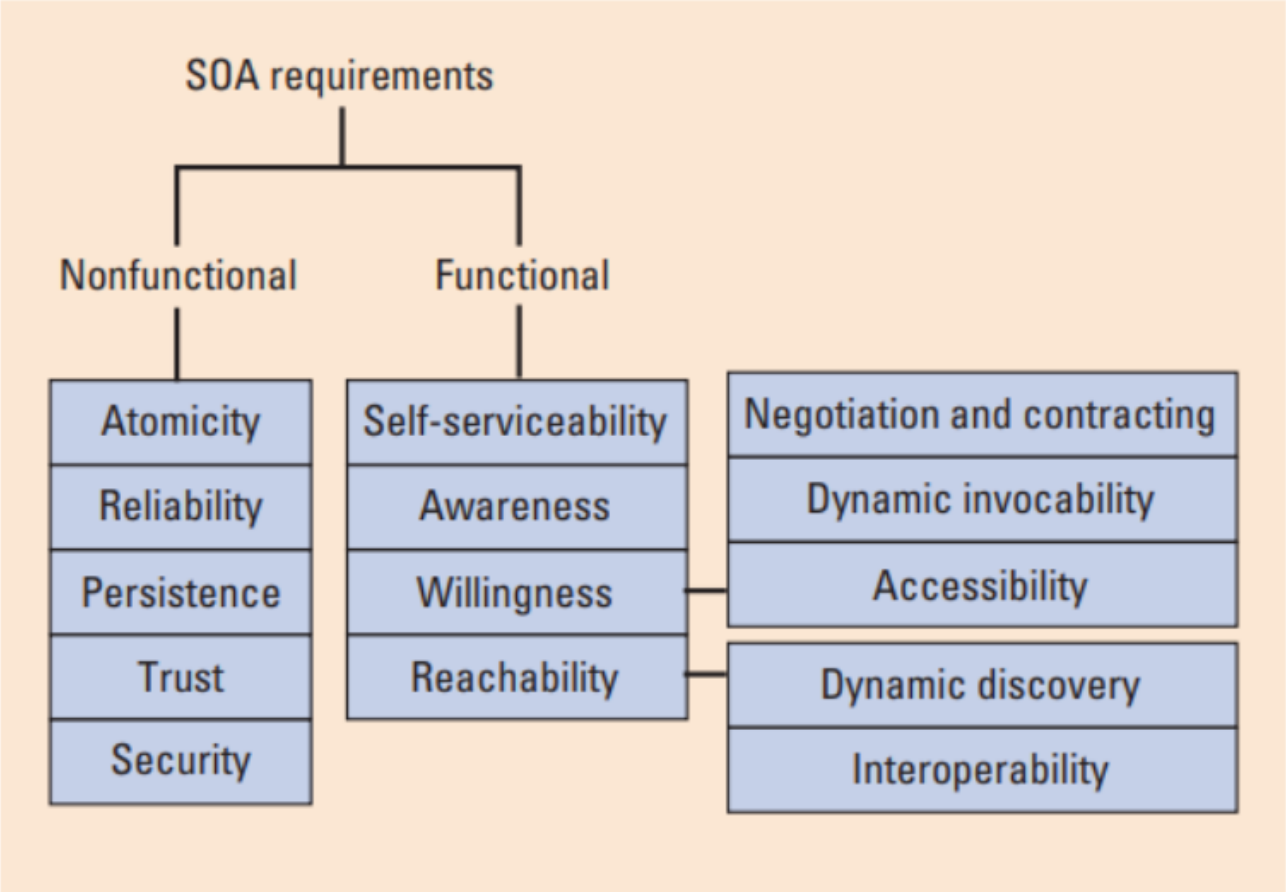

Figure 2. SOA requirement attributes can be decomposed into functional and nonfunctional requirements.

From another perspective, the Organization for the Advancement of Structured Information Standards (OASIS) defines SOA as "a paradigm for organizing and utilizing distributed capabilities that may be under the control of different ownership domains."2
This SOA description features three dimensions: visibility, interaction, and effect.

[**Definition:** A Web service is a software system designed to support interoperable machine-to-machine interaction over a network. It has an interface described in a machine-processable format. Other systems interact with the Web service in a manner prescribed by its description using messages, typically conveyed using HTTP with a data serialization method in conjunction

with other Web-related standards.] **https://www.w3.org/TR/ws-arch/#id2277502 (https://www.w3.org/TR/ws-arch/#id2277502)**

A Web Service is an abstract notion that must be implemented by a concrete agent. The agent is the concrete piece of software or hardware that sends and receives messages, while the service is the resource characterized by the abstract set of functionality that is provided. To illustrate this distinction, you might implement a particular Web service using one agent one day (perhaps written in one programming language), and a different agent the next day (perhaps written in a different programming language) with the same functionality. Although the agent may have changed, the Web service remains the same.

Web services provide a standard means of interoperating between different software applications, running on a variety of platforms and/or frameworks.

The Web services architecture is an *interoperability* architecture: it identifies those global elements of the global Web services network that are required in order to ensure interoperability between Web services.

Distributed applications in a Web services architecture communicate via message exchanges. These message exchanges are logically factored into patterns that may be composed at different levels to form larger patterns. A Message Exchange Pattern (MEP) is a template, devoid of application semantics, that describes a generic pattern for the exchange of (one-way) messages between agents. The patterns can be described by state machines that define the flow of the messages, including the handling of faults that may arise, and the correlation of messages.

A message is the basic unit of data sent from one Web services agent to another in the context of Web services.

A message represents the data structure passed from its sender to its recipients. The structure of a message is defined in a service description.

A message can be as simple as an HTTP GET request, in which the HTTP headers are the headers and the parameters encoded in the URL are the content. Note that extended Web services functionality in this architecture is not supported in HTTP headers.

A message can also simply be a plain XML document. However, such messages do not support extended Web services functionality defined in this architecture.

A message can be a SOAP XML, in which the SOAP headers are the headers. Extended Web services functionality are supported in SOAP headers.